

Zero to Hero Study Handbook: Production AI Resume Screener

Module 1: Foundations & Architecture

1.1 What this project does

This repository implements a production-style AI resume screening platform with four primary interfaces:

- FastAPI backend (`src/resume_ai/api/main.py`)
- Streamlit dashboard (`src/resume_ai/ui/app.py`)
- Typer CLI (`src/resume_ai/cli/main.py`)
- MCP server (`src/resume_ai/mcp/server.py`)

All interfaces call one facade service class: `ResumeAIService` in `src/resume_ai/service.py`.

Core use cases implemented in code:

- Ingest resume files (single file or batch folder) and parse them into structured schema.
- Detect digital vs scanned inputs and apply OCR when required.
- Store candidate/job/score/report/interview entities in SQLite.
- Store/search resume and JD embeddings in ChromaDB.
- Parse job descriptions, normalize skills, and compute explainable weighted candidate-job scores.
- Generate interview question packs and hiring report artifacts (`.md`, `.html`, `.json`, `.pdf`).
- Run recruiter semantic search with citations over indexed resume text.

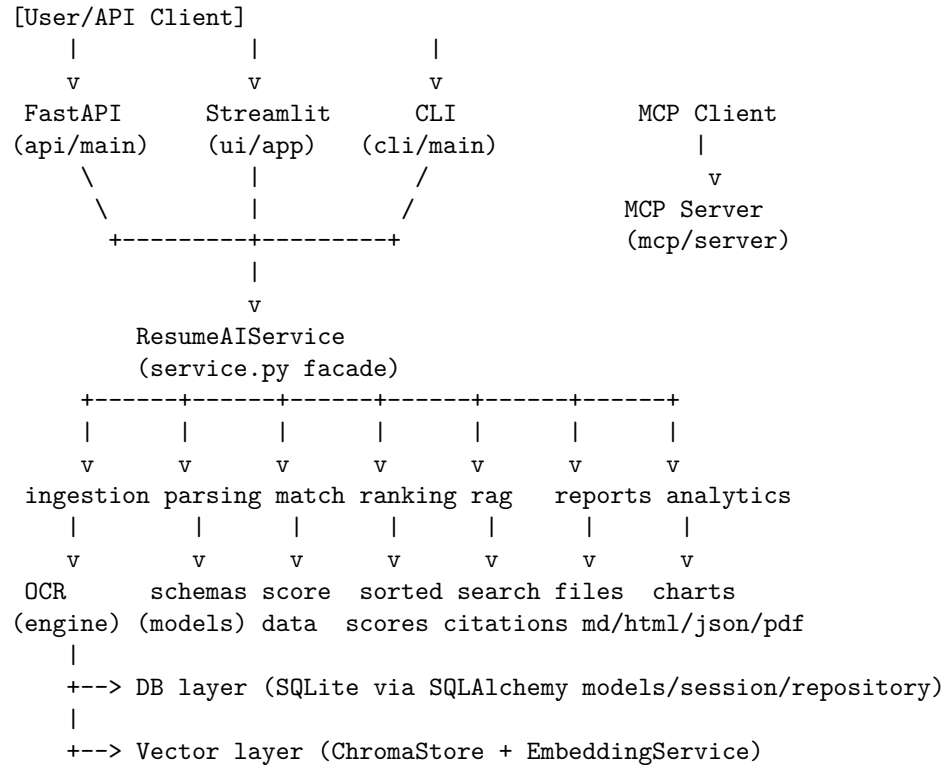
1.2 Core paradigms and patterns used here

- Facade pattern:
 - `ResumeAIService` centralizes orchestration and is reused by API/UI/CLI/MCP.
- Layered architecture:
 - `ingestion`, `parsing`, `matching`, `ranking`, `rag`, `reports`, `db`, `vector` modules are separated by responsibility.
- OOP with typed data contracts:
 - Pydantic models in `src/resume_ai/models.py` define canonical schemas (`ResumeParseResult`, `JobRequirementProfile`, `ScoreBreakdown`, etc.).
- Pipeline style processing:
 - `IngestionPipeline.ingest_file()` runs `read -> dedupe -> parse -> persist -> embed -> vector-index`.
- Adapter/wrapper pattern for infrastructure:
 - `ChromaStore` wraps Chroma operations.
 - `OllamaLLM` wraps Ollama HTTP calls.

- `EmbeddingService` switches between `SentenceTransformer` and `Ollama` fallback.
- Deterministic + LLM hybrid extraction:
 - Rule-based parsing first, then LLM JSON fixup (`ResumeParser`, `JobDescriptionParser`).
- Explainable scoring design:
 - `MatchingEngine` returns component scores, matched skills, missing skills, evidence, and confidence.

1.3 Architecture and component interaction

Primary runtime graph (ASCII):



Main storage backends in code:

- SQLite database path comes from `config/runtime/default.yaml` key `sqlite_path`.
- Chroma persistent directory comes from `config/runtime/default.yaml` key `chroma_path`.

Module 2: Repository Map

The table below prioritizes files a new contributor should read first.

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
pyproject.toml	Project metadata, dependencies, scripts	project.scripts entrypoints (resume-ai, resume-ai-api, resume-ai-dashboard, resume-ai-mcp)	requires-python >=3.12, dependency list, tool.pytest.ini_options
src/resume_ai/service.py	Core application fa- cade/orchestrator	ResumeAIService, upload_resume, create_job, score, search, generate_report	config_path, session_factory, component instances
src/resume_ai/models.py	Classical data contracts	ResumeParseResult, JobRequirementProfile, ScoreBreakdown, InterviewQuestionSet, APIEnvelope	Field names and defaults for all structured I/O
src/resume_ai/api/main.py	FastAPI routes and HTTP orchestration	health, upload, get_resume, score, compare, search, interview, reports, mcp_tool	app = FastAPI(...), envelope response contract
src/resume_ai/api/schemas.py	Asynchronous payload schemas	UploadRequest, JobRequest, ScoreRequest, CompareRequest, SearchRequest	blind_mode, top_k, weight_override, output_dir
src/resume_ai/cli/main.py	Train CLI commands	ingest, score, compare, interview, search, report, dashboard	command names and positional args
src/resume_ai/ui/app.py	Sphinxlit multi-page dashboard	run, _dashboard, _upload, _candidate_explorer, _ranking, _comparison, _interview, _analytics, _reports	PAGES list, UI labels, default IDs

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/resume_ai/config/llm.py	YAML loader, loading and validation	AppConfig, load_config, _deep_merge_dicts	RuntimeConfig, ModelConfig, EmbeddingConfig, OCRConfig, ScoringConfig, RetryConfig
config/config.yaml	Global config defaults mapping	YAML defaults list	active config groups (runtime, models, embeddings, ocr, scoring, prompts)
config/models/default.yaml	Multi-LLM model selection	N/A	extraction_model, reasoning_model, interview_model, scoring_model, fallback_model, parser_model, ollama_base_url
config/scoring/default.yaml	Weighted scoring profile	N/A	technical_skills, experience, projects, education, certifications, communication, bonus_skills
src/resume_ai/ingestion/pipeline.py	Document pipeline	IngestionPipeline, ingest_folder	ingest_files, ingest_folder, blind mode, handling, vector upsert metadata
src/resume_ai/ingestion/reader.py	Estimation/reader, OCR routing	ResumeReader.read	SUPPORTED_EXTENSIONS, _pdf_text_density, PDF density, compute_file_hash, threshold 0.002
src/resume_ai/ocr/ocr_engine.py	OCR engine, processing (Tesseract + optional Paddle)	OCREngine.extract_from_image, extract_from_pdf, preprocess_image	enable_paddle, tesseract_lang, dpi
src/resume_ai/parser/resume_parser.py	Single resume parsing, extraction	ResumeParser.parse, _parse_with_rules, _llm_fixup, _extract_skills	regex constants (EMAIL_RE, PHONE_RE, etc.), SKILL_KEYWORDS

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/resume_ai/parsing/jd_parser.py	Single JD parser and normalization	JobDescriptionParser, _parse_rules, _normalize	ABTASAME, SKILL_HINTS
src/resume_ai/parsing/redaction.py	Bing-hedaction redaction	redact_text	regex replacement tokens ([REDACTED_EMAIL], etc.)
src/resume_ai/matching/engine.py	Matching engine, weighted scoring	MatchingEngine.score_candidate, and component score methods	original candidate, semantic score blend, evidence construction
src/resume_ai/rag/assistant.py	Recruiter search and grounded answer	RecruiterAssistant, answer	Chunk candidates, fields (ids, documents, metadatas, distances)
src/resume_ai/vector/chroma_store.py	Chroma persistence/query wrapper	insert_resume, upsert_job, query_resumes	collection names (resume_chunks, job_descriptions, etc.)
src/resume_ai/db/schema.py	SQLAlchemy database schema	Candidate, Resume, JobDescription, CandidateJobScore, etc.	table names, JSON fields, indexes
src/resume_ai/db/operations.py	Database operations	upsert_candidate, insert_resume, sync_candidate_details, save_score	skill type mapping, detail tables, sync behavior
src/resume_ai/reports/generator.py	Reports generator implementation	export_markdown, export_json, export_html, export_pdf	output artifact paths and content structures
src/resume_ai/mcp/MCPTool.py	MCP tool exposure	build_server, MCP tool handlers	tool names (resume_search, candidate_lookup, etc.)
tests/unit/*.py, tests/integration/*.py	Behavior checks and message examples	parser/matching/deduplication test functions	expected API/CLI contracts asserted in tests

data/samples/sample_resume.txt, N/A	real example
data/samples/sample_jobs.txt	fields and domain
learning flows	vocabulary

Module 3: Core Execution Flows

3.1 Boot flow and dependency wiring

Primary service construction (`ResumeAIService.__init__`):

1. Load config: `load_config(config_path)`.
2. Build SQLAlchemy engine: `create_engine_from_config(self.config)`.
3. Create session factory: `get_session_factory(self.engine)`.
4. Create tables automatically: `Base.metadata.create_all(self.engine)`.
5. Instantiate feature modules: `IngestionPipeline`, `JobDescriptionParser`, `MatchingEngine`, `InterviewGenerator`, `RecruiterAssistant`, `EmbeddingService`, `ChromaStore`, `ProcessingQueue`.

Code fragment from `src/resume_ai/service.py`:

```
self.config = load_config(config_path)
self.engine = create_engine_from_config(self.config)
self.session_factory = get_session_factory(self.engine)
Base.metadata.create_all(self.engine)
```

3.2 Resume ingestion flow (/upload, CLI ingest, UI Upload page)

Main path is `ResumeAIService.upload_resume()` -> `IngestionPipeline.ingest_file()`.

Step-by-step:

1. `ResumeReader.read(path)` chooses extraction route by extension:
 - `.txt/.md`: direct text.
 - `.docx`: python-docx paragraph join.
 - image extensions: OCR (`extract_from_image`), mode `scanned`.
 - `.pdf`: text density check `_pdf_text_density`; if density > 0.002 use native text extraction, else OCR.
2. Compute SHA256 hash via `ResumeReader.compute_file_hash()`.
3. Exact duplicate check: `find_exact_duplicate(session, file_hash)`.
 - If duplicate exists, reuse stored `parsed_json` and sync denormalized detail tables.
4. Parse resume text: `ResumeParser.parse(text, ocr_mode, blind_mode)`.
5. Upsert candidate by email: `upsert_candidate(...)`.
6. Insert resumes row: `insert_resume(...)`.
7. Sync candidate_skills, experience, projects tables via `sync_candidate_details(...)`.
8. Build embedding with `EmbeddingService.embed_text(candidate_text)` where `candidate_text` = `parsed.redacted_text` or `text`.

9. Near-duplicate check: `find_near_duplicate(...)`; if matched, mark `resume.ingestion_status = "near_duplicate"`.
10. Vector upsert to Chroma: `ChromaStore.upsert_resume(...)` with meta-data keys:
 - `candidate_id, resume_id, ocr_mode, blinded`.

Key input shape:

- API request `UploadRequest`:

```
{
  "file_path": "data/samples/sample_resume.txt",
  "blind_mode": true
}
```

Key output shape (`upload_resume` return):

```
{
  "candidate_id": 1,
  "resume_id": 1,
  "parsed": {
    "candidate": {},
    "education": [],
    "experience": [],
    "projects": [],
    "skills": {},
    "certifications": [],
    "awards": [],
    "publications": [],
    "languages": [],
    "summary": "",
    "ocr_mode": "digital",
    "redacted_text": "..."
  }
}
```

3.3 Resume parser internal flow

`ResumeParser.parse()` in `src/resume_ai/parsing/resume_parser.py`:

1. Run `_parse_with_rules(text, ocr_mode)`:
 - Extract contact fields using regex constants.
 - Split content by section markers using `_sectionize(lines)`.
 - Build `EducationItem`, `ExperienceItem`, `ProjectItem` lists.
 - Build `SkillSet` using `SKILL_KEYWORDS` token matching.
2. Run `_llm_fixup(rule_result, text)`:
 - Prompt includes JSON schema from `ResumeParseResult.model_json_schema()`.
 - Calls `OllamaLLM.generate_json(model=self.config.models.parser_model)`.
 - Falls back to rule result on empty or validation error.

3. If `blind_mode=True`:
 - `redacted_text = redact_text(text)`.
 - Force `candidate.name = None` and `candidate.location = None`.

3.4 Job parsing and normalization flow (/job)

`ResumeAIService.create_job(jd_text)`:

1. Parse JD: `JobDescriptionParser.parse(jd_text)`.
2. Persist job row: `upsert_job(...)`.
3. Embed JD text: `EmbeddingService.embed_text(jd_text)`.
4. Upsert to Chroma `job_descriptions` collection.

Normalization details in `JobDescriptionParser`:

- Uses `ALIAS_MAP` (examples: `reactjs -> react`, `k8s -> kubernetes`, `llm -> large language models`).
- Extracts `required_skills`, `preferred_skills`, `responsibilities`, `experience_requirements`, `education_requirements`, `keywords`, `technologies`, `soft_skills`.

Key output shape:

```
{
  "job_id": 7,
  "parsed": {
    "title": "Senior Generative AI Engineer",
    "required_skills": [],
    "preferred_skills": [],
    "experience_requirements": [],
    "education_requirements": [],
    "responsibilities": [],
    "keywords": [],
    "technologies": [],
    "soft_skills": []
  }
}
```

3.5 Candidate scoring flow (/score, CLI score)

`ResumeAIService.score(candidate_id, job_id, weight_override)`:

1. Load latest resume for candidate and target job row.
2. Validate JSON into typed models:
 - `ResumeParseResult.model_validate(resume.parsed_json)`
 - `JobRequirementProfile.model_validate(job.parsed_json)`
3. `MatchingEngine.score_candidate(...)` computes:
 - `technical_skills` (required skill overlap + semantic blend)

- `experience, projects, education, certifications, communication, bonus_skills`
4. Weighted total uses config weights (`ScoringConfig`) or `weight_override`.
 5. Persist score row via `save_score(session, breakdown)`.
 6. Return `ScoreBreakdown`.

Key `ScoreBreakdown` fields from `src/resume_ai/models.py`:

- IDs: `candidate_id, job_id`
- Scores: `total_score, technical_skills, experience, projects, education, certifications, communication, bonus_skills`
- Explainability: `matched_skills, missing_skills, evidence[], confidence, weight_profile`

3.6 Ranking and comparison flow (/compare, UI Ranking/Comparison)

- Ranking path: `rank_for_job(job_id)`
 - Reads all `candidate_job_scores.breakdown_json` for the job.
 - Validates each as `ScoreBreakdown`.
 - Sorts by `total_score` descending via `rank_scores()`.
- Comparison path: `compare(job_id, candidate_ids)`
 - Reads score rows for job.
 - Filters to selected candidate IDs via `compare_candidates()`.
 - Sorts selected set with same ranking logic.

3.7 Semantic recruiter search flow (/search, CLI search)

`ResumeAIService.search(query, top_k) -> RecruiterAssistant.answer(...):`

1. Embed query with `EmbeddingService.embed_text(query)`.
2. Query Chroma resumes collection: `ChromaStore.query_resumes(query_embedding, n_results=top_k)`.
3. Build hit rows from Chroma payload keys:
 - `ids[0], documents[0], metadatas[0], distances[0]`.
4. Resolve candidate names from SQLite (`candidates` table).
5. Return:
 - `answer` string summarizing top matches.
 - `citations` list with `candidate_id, candidate_name, snippet`.

3.8 Interview generation flow (/interview, CLI interview)

`ResumeAIService.generate_interview(candidate_id, job_id):`

1. Recompute score via `self.score(...)`.
2. Generate question set via `InterviewGenerator.generate(score)`.
3. Persist to `interviews.questions_json` table as `InterviewPack`.
4. Return `InterviewQuestionSet.model_dump(mode="json")`.

Question sections produced in code:

- technical (from `missing_skills[:5]`)
- behavioral
- coding
- project

Each item includes `category`, `difficulty`, `question`, `follow_up`.

3.9 Report generation flow (/reports, CLI report)

`ResumeAIService.generate_report(candidate_id, job_id, output_dir):`

1. Compute fresh score.
2. Load latest parsed resume.
3. Build recommendation payload with `build_recommendation(parsed, score)`.
4. Create output directory.
5. Export four artifacts using:
 - `export_markdown(...)`
 - `export_html(...)`
 - `export_json(...)`
 - `export_pdf(...)`
6. Persist four `reports` table rows with `report_type` and `artifact_path`.
7. Return paths dictionary.

Output shape:

```
{
  "markdown": "outputs/reports/candidate_1_job_7.md",
  "html": "outputs/reports/candidate_1_job_7.html",
  "json": "outputs/reports/candidate_1_job_7.json",
  "pdf": "outputs/reports/candidate_1_job_7.pdf"
}
```

3.10 Batch queue flow (ingest on a folder)

- CLI `ingest(path)` detects folder and runs:
 - `enqueue_folder(path)` -> inserts `processing_jobs` rows (status=queued).
 - `run_queue(blind_mode, max_items):`
 - * `pop_next()` transitions row to `running` and increments attempts.
 - * Ingest file.
 - * On success: `mark_done()`.
 - * On exception: `mark_failed()` and save truncated `error_message`.

3.11 API envelope and route contract

Every API handler wraps output in `APIEnvelope` with structure:

```
{
  "status": "ok",
  "data": {},
  "errors": [],
  "trace_id": "uuid"
}
```

Routes defined in `src/resume_ai/api/main.py`:

- GET /health
- POST /upload
- GET /resume/{resume_id}
- GET /candidate/{candidate_id}
- POST /job
- POST /score
- POST /compare
- POST /search
- POST /interview
- POST /reports
- GET /reports
- GET /analytics
- POST /notes
- POST /mcp/tool

Module 4: Setup & Run Guide

This section is static-analysis-derived from repository files and does not assume any runtime was executed in this study pass.

4.1 Prerequisites

From `pyproject.toml` and code imports:

- Python `>=3.12`
- `uv` for environment/package management
- Local Ollama service reachable at URL configured by `models.ollama_base_url` (default `http://localhost:11434`)
- Tesseract OCR binary available for `pytesseract` runtime OCR paths

4.2 Install dependencies

```
uv venv .venv
source .venv/bin/activate
uv sync --extra dev
```

Optional OCR extras (from `[project.optional-dependencies].ocr`):

```
uv sync --extra ocr
```

4.3 Configuration files and key settings

Configuration root: `config/`

- `config/config.yaml`: Hydra-style defaults chain and app metadata.
- `config/runtime/default.yaml`:
 - `environment`, `sqlite_path`, `chroma_path`, `max_workers`, `batch_size`
- `config/runtime/retries.yaml`:
 - `llm_retries`, `timeout_seconds`
- `config/models/default.yaml`:
 - model routing and Ollama URL
- `config/embeddings/default.yaml`:
 - embedding backend/model and `output_dimension`
- `config/ocr/default.yaml`:
 - OCR toggles and OCR parameters
- `config/scoring/default.yaml`:
 - score component weights
- `config/prompts/default.yaml`:
 - prompt file mappings

Prompt definition files live in `src/resume_ai/prompts/*.yaml` and include objective, constraints, schema, and example sections.

4.4 Environment variables (.env keys)

Static code scan across `src/` shows no required environment variable reads via `os.getenv/os.environ`.

Implication:

- This codebase currently relies on YAML config values rather than `.env` keys.
- If you need different runtime endpoints or paths, update config YAML files.

4.5 Entrypoints and command sequences

From `[project.scripts]` in `pyproject.toml`:

- `resume-ai -> resume_ai.cli.main:app`
- `resume-ai-api -> resume_ai.api.main:run`
- `resume-ai-dashboard -> resume_ai.ui.app:run`
- `resume-ai-mcp -> resume_ai.mcp.server:run`

Typical command sequences:

API

`uv run resume-ai-api`

Dashboard

```
uv run resume-ai-dashboard
```

```
# CLI help
```

```
uv run resume-ai --help
```

```
# MCP server
```

```
uv run resume-ai-mcp
```

CLI workflow examples:

```
uv run resume-ai ingest data/samples/sample_resume.txt --blind-mode
```

```
uv run resume-ai score 1 1
```

```
uv run resume-ai search "LangGraph experience" --top-k 5
```

```
uv run resume-ai interview 1 1
```

```
uv run resume-ai report 1 1 --output-dir outputs/reports
```

```
uv run resume-ai compare 1 1
```

4.6 Database migration/seeding notes

- No Alembic migration scripts exist in this repo.
- Table creation is automatic at service startup via `Base.metadata.create_all(self.engine)`.
- Sample seed-like inputs available in:
 - `data/samples/sample_resume.txt`
 - `data/samples/sample_jd.txt`

4.7 Interface-specific notes

- API: FastAPI app object in `src/resume_ai/api/main.py`.
- UI: page map and controls in `src/resume_ai/ui/app.py` (PAGES list).
- CLI: command names and signatures in `src/resume_ai/cli/main.py`.
- MCP: tool names and signatures in `src/resume_ai/mcp/server.py`.

Module 5: Study Plan & Practice Exercises

5.1 Ordered study plan

Use this reading order to build understanding incrementally.

1. Foundation contracts:
 - `pyproject.toml`
 - `src/resume_ai/models.py`
 - `src/resume_ai/config/loader.py` and all `config/*.yaml`
2. Service orchestration:
 - `src/resume_ai/service.py`
3. Data and persistence:
 - `src/resume_ai/db/models.py`
 - `src/resume_ai/db/session.py`
 - `src/resume_ai/db/repository.py`

4. Ingestion + parsing stack:
 - `src/resume_ai/ingestion/readers.py`
 - `src/resume_ai/ocr/engine.py`
 - `src/resume_ai/parsing/resume_parser.py`
 - `src/resume_ai/parsing/jd_parser.py`
 - `src/resume_ai/parsing/redaction.py`
5. Intelligence layer:
 - `src/resume_ai/embeddings/service.py`
 - `src/resume_ai/vector/chroma_store.py`
 - `src/resume_ai/matching/engine.py`
 - `src/resume_ai/ranking/engine.py`
 - `src/resume_ai/rag/assistant.py`
6. User interfaces:
 - `src/resume_ai/api/main.py`
 - `src/resume_ai/cli/main.py`
 - `src/resume_ai/ui/app.py`
 - `src/resume_ai/mcp/server.py`
7. Artifact/reporting/ops:
 - `src/resume_ai/interview/generator.py`
 - `src/resume_ai/reasoning/recommender.py`
 - `src/resume_ai/reports/generator.py`
 - `src/resume_ai/analytics/metrics.py`
 - `src/resume_ai/observability/*.py`
 - `src/resume_ai/workers/queue.py`
8. Validation examples:
 - `tests/unit/*.py`
 - `tests/integration/*.py`

5.2 Practice exercises

Exercise 1 Explain the exact ingestion decision logic for a `.pdf` in `ResumeReader.read()`.

Exercise 2 List every table that receives writes during a successful `upload_resume()` call for a non-duplicate file.

Exercise 3 Trace how blind hiring mode changes parsed output fields and stored text.

Exercise 4 Given a custom score weight override payload, identify where it is applied and how defaults are merged.

Exercise 5 Describe the data flow of `/search` from request payload to response citations.

Exercise 6 Identify all interfaces that can trigger `generate_report()` and list all artifact formats it creates.

Exercise 7 Find where near-duplicate logic is implemented and explain what status change is made when a near match is detected.

Exercise 8 Compare API and CLI score outputs: what type each function returns before serialization, and where conversion happens.

Exercise 9 Explain how MCP tools map to underlying `ResumeAIService` methods.

Exercise 10 Add a hypothetical new metric called `median_match_score`: which file(s) would you edit and why?

5.3 Solution outlines

Solution 1 In `src/resume_ai/ingestion/readers.py`, `.pdf` files call `_pdf_text_density(path)`. If `density > 0.002`, use direct text extraction via `PyMuPDF` and return `OCRMode.DIGITAL`; otherwise call `OCREngine.extract_from_pdf(path)` and return `OCRMode.SCANNED`.

Solution 2 For non-duplicate upload:

- `candidates` via `upsert_candidate`
- `resumes` via `insert_resume`
- `candidate_skills`, `experience`, `projects` via `sync_candidate_details`
- Chroma `resume_chunks` collection via `ChromaStore.upsert_resume`

If duplicate is exact, no new `resumes` row is inserted; details are resynced from stored parse.

Solution 3 `ResumeParser.parse(..., blind_mode=True)` sets:

- `fixed.redacted_text = redact_text(text)`
- `fixed.candidate.name = None`
- `fixed.candidate.location = None`

Redaction function replaces email/phone/links with `[REDACTED_*]` tokens.

Solution 4 `ScoreRequest.weight_override` reaches `ResumeAIService.score(..., weight_override=...)`, then `MatchingEngine.score_candidate(..., override_weights=...)`. `_resolve_weights()` starts with `self.config.scoring.model_dump()`, updates with override dict, then validates into `ScoringConfig`.

Solution 5 `/search` route accepts `SearchRequest(query, top_k)`, calls `ResumeAIService.search`, then `RecruiterAssistant.answer(session, query, top_k)`:

- embed query
- Chroma query over `resume_chunks`
- collect candidate IDs/snippets/distances
- resolve candidate names from SQLite
- return `{answer, citations}`

Solution 6 `generate_report()` can be called by:

- API route `POST /reports`
- CLI command `resume-ai report`
- Streamlit Reports page button
- MCP tool `generate_report`
- HTTP bridge `/mcp/tool` with `tool == "generate_report"`

Artifacts produced: markdown, html, json, pdf.

Solution 7 Near-duplicate logic:

- `find_near_duplicate()` in `src/resume_ai/ingestion/dedupe.py`
- Called in `IngestionPipeline.ingest_file()` after embedding.
- If near duplicate found and ID differs, `resume.ingestion_status = "near_duplicate"`.

Solution 8 `ResumeAIService.score()` returns `ScoreBreakdown` model object.

- API converts with `out.model_dump(mode="json")` before envelope.
- CLI converts with `result.model_dump_json()` for `console.print_json`.

Solution 9 In `src/resume_ai/mcp/server.py`:

- `resume_search -> service.search`
- `candidate_lookup -> service.get_candidate`
- `generate_interview -> service.generate_interview`
- `score_candidate -> service.score(...).model_dump(...)`
- `generate_report -> service.generate_report`

Solution 10 `median_match_score` belongs in analytics aggregation:

- compute in `src/resume_ai/analytics/metrics.py` (`build_snapshot`),
- add field to `AnalyticsSnapshot` in `src/resume_ai/models.py`,
- surface in API/UI via `service.analytics()` and dashboard rendering code.

Understanding Checklist

Use this checklist after studying.

- ☐ Can you explain how **ResumeAIService** wires config, DB, ingestion, matching, RAG, and reporting modules?
- ☐ Can you trace a **POST /upload** request end-to-end into SQLite and Chroma writes?
- ☐ Can you explain how digital vs scanned PDF detection works and where OCR is invoked?
- ☐ Can you describe the exact fields of **ResumeParseResult**, **JobRequirementProfile**, and **ScoreBreakdown**?
- ☐ Can you explain how weighted scoring is computed and where explainability evidence is generated?
- ☐ Can you explain how recruiter search builds citations from Chroma and candidate name lookup?
- ☐ Can you list all command/API/UI/MCP entrypoints that invoke report generation?
- ☐ Can you explain the queue lifecycle states (**queued**, **running**, **done**, **failed**) in **processing_jobs**?
- ☐ Can you identify where to modify model selection, scoring weights, and OCR settings in YAML config?
- ☐ Can you propose one extension (feature or metric) and identify exact files to change?